

Ejercicios Guiados para programar en Rstudio: Metropolis-Hastings

Parte I: Entendiendo la Caminata Aleatoria

Ejercicio 1: Caminata Aleatoria Unidimensional

Considere una caminata aleatoria simple en $\mathbb{R}$:

$$\theta^{(m+1)} = \theta^{(m)} + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2) \quad (1)$$

- (a) Simule 1000 pasos de esta caminata con $\theta^{(0)} = 0$ y $\sigma = 0.1$.
- (b) Grafique la trayectoria. ¿Qué observa?
- (c) Repita con $\sigma = 1.0$. Compare ambas trayectorias.
- (d) **Reflexión:** ¿Cómo afecta σ a la exploración del espacio?

Ejercicio 2: Caminata Aleatoria multidimensional

Repetir el ejercicio anterior en dimensiones mayor que 1.

Ejercicio 3: Modelo Poisson con Metropolis-Hastings

Implementar el algoritmo de Markov Chain Monte Carlo (MCMC) basado en Metropolis Hastings para estimar el número esperado de accidentes de avión fatales en un año, θ , y la probabilidad de que ocurran 30 accidentes fatales en un solo año $\mathbb{P}(Y > 30|\theta)$.

Este problema se plantea en el siguiente enlace.

[https://www.y1zhou.com/series/bayesian-stat/bayesian-stat-bayesian-inference-?utm_source](https://www.y1zhou.com/series/bayesian-stat/bayesian-stat-bayesian-inference/?utm_source)

1. Usando el modelo Poisson con prior no-informativo

2. Repetir el ejercicio anterior pero ahora usando los priors $\text{Gamma}(3,2)$ y $\text{Gamma}(9, 0.5)$.
3. Comparar las densidades posteriores: histograma, ajuste-empírico, y teórica (poisson-informativo, poisson-gamma).
4. Modificar el tamaño de pasos en la caminata aleatoria para obtener una aceptación entre 30 % y 40 %.
5. Visualización de resultados (trace plot, posteriores, etc.)

Ver codigos en la carpeta mcmc del git-repositorio

Nota: en clases anteriores se demostró que la distribución posterior para un modelo Poisson con prior constante 1 es una distribución Gamma: $\lambda|\mathbf{y} \sim \text{Gamma}(\sum_{i=1}^n y_i + 1, n)$

Ejercicio 4: Modelo Binomial negativa con Metropolis-Hastings

Repetir el ejercicio anterior pero ahora modelamos los datos con una Negativa binomial, i.e.,

$$Y_k \sim \text{NegBinom}\left(\frac{\mu}{\mu + r}, r\right), \quad (2)$$

donde

$$\mathbb{P}[Y_k = y_k] = \binom{y_k + r - 1}{y_k} \left(\frac{\mu}{\mu + r}\right)^{y_k} \left(\frac{r}{\mu + r}\right)^r \quad (3)$$

$$= \frac{\Gamma(y_k + r)}{y_k! \Gamma(r)} \left(\frac{\mu}{\mu + r}\right)^{y_k} \left(\frac{r}{\mu + r}\right)^r \quad (4)$$

Aquí, el parametro μ es el valor esperado del número de accidentes y r es el parámetro de dispersión. Equivalentemente, la probabilidad de éxito es

$$p = \frac{r}{r + \mu}$$

https://en.wikipedia.org/wiki/Negative_binomial_distribution

Por lo que el log del likelihood function es

$$l(\mu) \propto \sum \log(\Gamma(y_k + r)) - n \log(\Gamma(r)) + \sum_{k=1}^n y_k \log\left(\frac{\mu}{\mu + r}\right) + r \log\left(\frac{r}{r + \mu}\right)$$

$$l(\mu) \propto \sum \log(\Gamma(y_k + r)) - n \log(\Gamma(r)) + \sum_{k=1}^n y_k \log(1 - p) + r \log(p)$$

To do: escribir como generar el likelihood directo en R y revisar eficiencias, time-cost, verificar que las fórmulas esten bien escritas, ver wikipedia página?

Ejercicio 4: De Caminata Aleatoria a Metropolis-Hastings

Contexto: Tenemos datos $y = \{6.2, 4.8, 5.1, 4.9, 5.3\}$ de una distribución $\mathcal{N}(\mu, 1)$ con varianza conocida. Queremos estimar μ .

Configuración Bayesiana:

$$\text{Verosimilitud: } y_i \sim \mathcal{N}(\mu, 1) \quad (5)$$

$$\text{Prior: } \mu \sim \mathcal{N}(0, 10) \quad (\text{prior débil}) \quad (6)$$

$$\text{Posterior: } \mu|y \sim \mathcal{N}(\mu_{post}, \sigma_{post}^2) \quad (7)$$

donde (por conjugación):

$$\mu_{post} = \frac{10 \cdot \bar{y} + 1 \cdot 0}{10 + 1} = \frac{10\bar{y}}{11} \quad (8)$$

$$\sigma_{post}^2 = \frac{1 \cdot 10}{10 + 5} = \frac{10}{15} = \frac{2}{3} \quad (9)$$

(a) **Paso 1:** Calcule analíticamente μ_{post} y σ_{post}^2 con los datos dados.

(b) **Paso 2:** Implemente la función de log-densidad posterior:

$$\ell(\mu) = \sum_{i=1}^n \left[-\frac{1}{2}(y_i - \mu)^2 \right] - \frac{1}{2 \cdot 10} \mu^2 + \text{const} \quad (10)$$

(c) **Paso 3:** Compare tres enfoques para generar muestras:

- **Caminata pura:** Acepta todos los pasos $\mu^{(m+1)} = \mu^{(m)} + \epsilon$
- **Metropolis-Hastings:** Usa mecanismo de aceptación/rechazo
- **Muestreo directo:** Muestrea directamente de $\mathcal{N}(\mu_{post}, \sigma_{post}^2)$

(d) **Paso 4:** Para MH, compare tasas de aceptación con $\sigma_{prop} \in \{0.1, 0.5, 1.0, 2.0\}$.

(e) **Pregunta clave:** ¿Por qué MH converge a la posterior correcta mientras que la caminata pura no?

Parte II: Casos Multidimensionales

Ejercicio 3: Normal Bivariada

Objetivo: Muestrear de $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ con:

$$\boldsymbol{\mu} = \begin{pmatrix} 2 \\ 3 \end{pmatrix}, \quad \boldsymbol{\Sigma} = \begin{pmatrix} 1 & 0.5 \\ 0.5 & 2 \end{pmatrix} \quad (11)$$

Ejercicio 3: Normal Bivariada

Objetivo: Muestrear de $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ con:

$$\boldsymbol{\mu} = \begin{pmatrix} 2 \\ 3 \end{pmatrix}, \quad \boldsymbol{\Sigma} = \begin{pmatrix} 1 & 0.5 \\ 0.5 & 2 \end{pmatrix} \quad (12)$$

- (a) Use matriz de propuesta $\boldsymbol{\Sigma}_{prop} = c \cdot \mathbf{I}_2$ con $c \in \{0.1, 0.5, 1.0, 2.0\}$.
- (b) Ejecute 10,000 iteraciones para cada valor de c .
- (c) Grafique las trayectorias y compare tasas de aceptación.
- (d) Calcule las medias y covarianzas muestrales. Compare con los valores teóricos.

Parte III: Transformaciones de Parámetros

Ejercicio 4: Parámetro con Restricción Positiva

Queremos muestrear de una distribución Gamma:

$$\theta \sim \text{Gamma}(\alpha = 2, \beta = 1), \quad \theta > 0 \quad (13)$$

- (a) **Enfoque directo:** Implemente MH directamente en $\theta > 0$, rechazando propuestas negativas.
- (b) **Con transformación:** Use $\xi = \log(\theta)$, implemente MH en $\xi \in \mathbb{R}$.
- (c) Compare eficiencia: tasas de aceptación, mezcla, tiempo de autocorrelación.

Parte IV: Aplicación Epidemiológica

Ejercicio 5: Modelo SIR Simplificado

Proyecto Final: Implementación Propia

Desarrolle su propia clase **MetropolisHastings** que incluya:

- Soporte para transformaciones automáticas de parámetros
- Calibración adaptativa de la matriz de propuesta durante burn-in
- Cálculo automático de diagnósticos de convergencia
- Visualización de resultados (trace plots, posteriores, etc.)
- Manejo robusto de errores numéricos

Pruebe su implementación en al menos tres problemas diferentes y compare con implementaciones estándar (ej. PyMC, Stan).